

Lösungsheft

Tag 10

Musterlösungen zu den elf Aufgaben
des Aufgabenhefts – Microservices,
Kubernetes, TDD und Operating Model.

Musterlösungen · nicht die einzige Antwort

Hinweis:

Musterlösungen sind Diskussionsbasis,
nicht die einzige korrekte Lösung.

Begleitend:

Aufgabenheft & Lehrskript Tag 10

Dozent:

Can Siebert

Niveau-Mix:

3 L1 · 5 L2 · 3 L3

Hinweise zu den Musterlösungen

Die folgenden Musterlösungen sind *eine* mögliche, didaktisch nachvollziehbare Lösung – sie sind nicht die einzig richtige. Auf L2 und L3 sind mehrere Begründungswege legitim, solange sie:

- den im Lehrskript eingeführten Begriffsapparat (Microservices, Kubernetes, TDD, DevOps, Operating Model) sauber nutzen,
- Annahmen explizit machen, statt sie zu verschleiern,
- Zielkonflikte (Speed vs. Stabilität, Autonomie vs. Standardisierung, Aufwand vs. Auditierbarkeit) benennen,
- zu einer klar formulierten Entscheidung führen – nicht bloß zu einer Beschreibung.

Nutzung im Coaching

Vergleichen Sie Ihre eigene Antwort *vor* der Musterlösung. Wo weicht Ihre Argumentation ab? Ist die Abweichung eine andere Annahme – oder ein Denkfehler? Genau dort entsteht der Lerneffekt.

Niveau L1 – Wissen & Reproduktion

Hilfsvideos zu diesem Tag

Die folgenden YouTube-Erklärvideos vermitteln die Inhalte, die Sie zur Bearbeitung der Aufgaben benötigen. Klicken Sie auf den Titel, um das Video direkt zu öffnen; die vollständige URL steht in Klammern.

1. Microservices — warum, wann und mit welchen Grenzen

<https://youtu.be/ZXfLy229GLs>

(URL: `https://youtu.be/ZXfLy229GLs`)

2. Kubernetes — Container-Orchestrierung in der Praxis

<https://youtu.be/SFVg7fDdvLE>

(URL: `https://youtu.be/SFVg7fDdvLE`)

3. Test-Driven Development — Qualität als Entscheidungsinstrument

<https://youtu.be/71nLhdZuMk0>

(URL: `https://youtu.be/71nLhdZuMk0`)

4. DevOps-Operating-Model — „you build it, you run it“ und Plattform-Teams

<https://youtu.be/ak3oE0I6cXU>

(URL: `https://youtu.be/ak3oE0I6cXU`)

Tipp: Öffnen Sie das Video, lassen Sie es im Hintergrund laufen und arbeiten Sie parallel an der Aufgabe.

Aufgabe 1 – Microservices – Eignung, Risiken und Servicegrenze

L1 • Wissen

~ 8 min

YouTube-Suchquery: Microservices Vorteile Risiken Servicegrenze Business Capability erklärt

Musterlösung

1. Drei Nutzenversprechen:

- *Unabhängige Deployments*: Teams releasen ihren Service ohne Koordination mit allen anderen – kürzere Lead Time, niedrigere Stillstandskosten.
- *Gezielte Skalierung*: Nur der Service, der unter Last steht, wird skaliert (z. B. *Checkout* zur Peak-Saison). Kein Skalieren “auf Verdacht”.
- *Klare Verantwortungsgrenzen*: Ein Service, ein Team, ein Owner. “Verantwortung verschwindet nicht im Korridor.”

2. Drei Risiken:

- *Komplexität*: Viele bewegliche Teile, Netzwerk als unzuverlässige Komponente, Latenzen, Versionierung der Schnittstellen.
- *Verteilte Fehler*: Ein Fehler wandert über Service-Grenzen; Debugging ist ohne saubere Korrelations-IDs sehr teuer.
- *Observability-Bedarf*: Logs, Metriken, Traces zentral und konsistent – sonst dauert die Root-Cause-Analyse bei jedem Incident Stunden.

3. Gute Servicegrenze: Eine Servicegrenze ist *gut*, wenn sie entlang einer fachlichen Capability verläuft (z. B. *Inventory*, *Billing*) und *nicht* entlang von Datenbanktabellen. Tabellen sind Implementierungsdetail; eine Capability ist die kleinste Einheit geschäftlicher Verantwortung – sie überlebt Refactorings und definiert sauberen Owner-Schnitt.

Aufgabe 2 – Container & Kubernetes – Kernbegriffe

L1 • Wissen

~ 8 min

YouTube-Suchquery: Kubernetes Grundlagen Container Orchestrierung Blue Green Canary einfach erklärt

Musterlösung

Begriff	Erklärung
Container	Verpackte Anwendung inklusive aller Abhängigkeiten – läuft auf jedem kompatiblen Host gleich. “Build once, run anywhere.”
Orchestrierung	Plattform-Layer, der Container automatisch startet, neu plant, vernetzt, skaliert und ausfällige neu hochzieht (Kubernetes als typische Implementierung).
Self-Healing	Wenn ein Container abstürzt, startet die Orchestrierung ihn automatisch neu – ohne menschliches Eingreifen.
Rollback	Rückkehr zu einer vorherigen, bekannten guten Version, wenn ein neues Release Fehler zeigt.
Service-Discovery	Services finden sich über logische Namen, nicht über IP-Adressen – Änderungen am Cluster brechen Aufrufer nicht.
Blue/Green-Deployment	Zwei vollständige Versionen laufen parallel; der Verkehr wird in einem Schritt von “blau” auf “grün” umgeschaltet.
Canary-Release	Eine kleine Menge des Verkehrs (z. B. 5 %) geht zur neuen Version; bei stabilen Kennzahlen wird der Anteil schrittweise erhöht.
Ressourcenlimits	Pro Container vereinbarte Mindest- (Requests) und Höchstwerte (Limits) für CPU/RAM – verhindert, dass ein Service den Cluster “frisst”.

Zwei kritischste: *Rollback* und *Ressourcenlimits*. Ohne geübten Rollback wird jeder Incident zur Improvisation; ohne Limits bringt ein einzelner fehlerhafter Service die Plattform in den Ressourcen-Tod. Beide kosten in der Praxis am schnellsten am meisten Geld.

Aufgabe 3 – Testpyramide & Quality Gates

L1 • Wissen

~ 10 min

YouTube-Suchquery: Testpyramide TDD Red Green Refactor Quality Gates erklärt

Musterlösung

Eigenschaft	Unit-Test	Integrationstest	E2E-Test
Was wird getestet?	Einzelne Funktion / Klasse isoliert.	Zusammenspiel von 2–3 Komponenten (z. B. Service + DB).	Ganzer Geschäftsfluss über mehrere Services.
Geschwindigkeit	Sehr schnell (Millisekunden).	Mittel (Sekunden).	Langsam (Minuten).
Kosten / Fragilität	Günstig, stabil.	Mittel, mäßig fragil.	Teuer, sehr fragil (UI, Netzwerk, Timing).
Anteil in Pyramide	Viele (~ 70 %).	Mittel (~ 20 %).	Wenige (~ 10 %).
Typischer Fehlerfund	Logikfehler in einer Methode.	Schnittstelle / Datenformat falsch.	Gehört zusammen, aber im Gesamtfluss bricht etwas.

Quality Gates sind verbindliche Prüfungen in der Pipeline, die jeder Build durchlaufen muss, bevor er weiter darf. Drei nicht verhandelbare Gates vor einem Produktiv-Release:

1. *Alle Unit- und Integrationstests grün.*
2. *Static Code- und Security-Scan ohne kritische Findings* (z. B. keine Secrets im Code, keine bekannten CVEs in Dependencies).
3. *Smoke Test nach Deployment* (mindestens ein “Happy Path”-Aufruf gegen die neue Version).

Niveau L2 – Anwendung & Transfer

Aufgabe 4 – Service-Schnitt aus Prozessbeschreibung – Order-to-Cash

L2 • Anwendung

~ 25 min

YouTube-Suchquery: Microservices Service Schnitt Order to Cash Domain Driven Design Capability

Musterlösung

1. Sechs Service-Kandidaten:

- *Order Service* – Bestellungen annehmen, validieren, im Lifecycle führen.
- *Inventory Service* – Verfügbarkeit prüfen, Reservierungen, Bestände.
- *Pricing/Quote Service* – Preise und Angebote berechnen.
- *Shipping Service* – Versandaufträge, Tracking.
- *Billing Service* – Rechnungen erzeugen, Versand, Mahnwesen.
- *Payments Service* – Zahlungen abwickeln, Rückbuchungen.
- *Complaints Service* – Reklamationen, Gutschriften, Eskalation.

2. Drei Service-Steckbriefe:

(a) Order Service. *Zweck:* Lebenszyklus einer Bestellung halten (angelegt → bestätigt → verschickt → abgeschlossen). *Input:* Kunde, Artikelpositionen, Lieferadresse, gewünschter Liefertermin. *Output:* Bestell-ID, Status, bestätigter Liefertermin, Gesamtsumme. *Owner:* Produktteam Order. *Kritischster Fehler:* Bestellung in inkonsistentem Zustand (bestätigt, aber Inventory nicht reserviert) – Doppelt-Versand oder Lieferversprechen ohne Deckung.

(b) Payments Service. *Zweck:* Zahlungen für bestätigte Bestellungen verarbeiten und buchen. *Input:* Bestell-ID, Betrag, Zahlungsmittel, IBAN/Karteninfo (via Token). *Output:* Payment-Status, Buchungs-ID, Beleg-Referenz, Fehlerursache. *Owner:* Produktteam Payments + Compliance. *Kritischster Fehler:* Doppelbuchung oder fehlgeschlagene Buchung ohne Storno – direkte finanzielle und reputative Folgen.

(c) Inventory Service. *Zweck:* Bestände, Reservierungen und Verfügbarkeitsprüfungen. *Input:* Artikel-ID, Menge, Standort, Reservierungszeitraum. *Output:* Verfügbarkeit (ja/nein/eingeschränkt), Reservierungs-ID, voraussichtliches Wiederverfügbarkeits-Datum, Sicherheitsbestand-Status. *Owner:* Produktteam Inventory. *Kritischster Fehler:* Verkauf von tatsächlich nicht vorhandenem Bestand ("Overselling").

3. Riskanteste Grenze + Guardrail: *Order – Inventory.* Hier kollidieren zwei Verantwortlichkeiten: Order *verspricht*, Inventory *deckt*. Datenhoheit ist häufig unklar (Reservierung als Order- oder Inventory-Konzept?). *Guardrail:* **Anti-Corruption-Layer + idempotente Reservierungs-API:** Order ruft Inventory mit eindeutiger Operations-ID; Inventory antwortet entweder "reserviert" (mit TTL) oder "nicht verfügbar". Doppelaufrufe wirken sich *nicht* mehrfach aus. Ein *gemeinsames Datenmodell* ist verboten – jede Seite hält ihre eigene Sicht.

Andere Schnitte sind legitim (z. B. Trennung Pricing in Quote + Pricing, oder Notifications als

eigener Service). Entscheidend: Capability-Logik, klarer Owner, niedrige Kopplung.

Aufgabe 5 – Release-Flow & Testpyramide als Sicherheitsnetz

L2 • Anwendung

~ 25 min

YouTube-Suchquery: Release Flow Quality Gates Canary Feature Toggle SLO Beispiel

Musterlösung

1. Minimaler Release-Flow (7 Schritte):

1. *Commit + PR* – Code-Review durch zweite Person.
2. *Build + Unit/Integrationtests* – **Quality Gate 1**: Tests grün, Coverage $\geq 80\%$.
3. *Static & Security Scan* – Dependency-Check, Secret-Scan, License-Check.
4. *Stage-Deployment + Smoke Test* – Happy Path und einen Fehlerpfad prüfen.
5. *Approval* – **Quality Gate 2**: Service-Owner gibt explizit frei (bei kritischen Services zusätzlich Security).
6. *Canary auf Prod* – 5 % Verkehr, 30 Minuten Beobachtung (Fehlerquote, Latenz, Logs).
7. *Full Rollout* bei stabilen Kennzahlen, sonst automatischer Rollback.

2. Testpyramide für drei Services:

Service	Unit	Integration	E2E
Order	~ 120 (Geschäftsregeln, Statuslogik)	~ 25 (DB, Inventory-Aufruf)	2 (Happy Path, Fehlerpfad)
Billing	~ 90 (Berechnung, Steuerlogik)	~ 20 (PDF-Erzeugung, Versand)	1 (Rechnung an Kunde)
Payments	~ 140 (PSP-Adapter, Storno-Logik)	~ 30 (Zahlungsprovider, Buchung)	2 (Erfolgsfall, Rückbuchung)

3. Zwei Betriebskennzahlen (SLO-orientiert):

- *Verfügbarkeit Order-API*: 99,9 % pro Monat. Owner: Service-Owner Order. *Messfenster*: rollierende 30 Tage.
- *Fehlerquote Payments*: < 0,5 % der Buchungsversuche. Owner: Service-Owner Payments + Plattformteam (Alerts). *Messfenster*: rollierende 7 Tage.

4. MVP-Risikoplan (Rest-Risiko + Kompensation):

- *Bewusster Verzicht*: weniger E2E-Tests (nur 1 – 2 Kernpfade pro Service).
- *Kompensation 1*: *Canary-Release* mit harten Alerting-Schwellen (Fehlerquote, p95-Latenz) – kein full rollout, solange Kennzahlen unklar.
- *Kompensation 2*: *Feature Toggles* für riskante Änderungen – Änderungen lassen sich ohne Deployment ausschalten.
- *Kompensation 3*: *Manuelle Stichprobe* für Zahlungen mit Beträgen $\geq 1.000\text{€}$ in den ersten zwei Wochen nach größeren Releases.

Aufgabe 6 – Fallstudie “InsureFast” – Servicegrenzen & Betriebskonzept

L2 • Anwendung

~ 30 min

YouTube-Suchquery: Microservices Versicherung Schadenmeldung Kubernetes Skalierung Beispiel

Musterlösung

1. Servicegrenzen (sieben Services):

- *Claim Intake* – Eingang aller Schadenmeldungen (App, Telefon, Mail). Validierung, Vorqualifizierung.
- *Policy/Customer* – Vertrags- und Kundendaten (Datenhoheit Kunde).
- *Fraud Check* – Betrugsindikatoren, automatisierte Prüfung, Flagging.
- *Claim Assessment* – Sachprüfung, Entscheidung, Begründung (auditrelevant!).
- *Payout* – Auszahlung, Verknüpfung mit Finance.
- *Notifications* – Mail, Push, SMS an Kunden + interne Stakeholder.
- *Document Management* – Beweisdokumente, Fotos, Gutachten, Archivierung.

Begründung einer nicht-trivialen Grenze: *Fraud Check* als eigener Service (nicht in *Claim Assessment*). Grund: *Fraud Check* ist regelmäßig algorithmisch-dynamisch (Modelle, Schwellenwerte), wird häufiger geändert und braucht eigene Datenhoheit (Score-Historie). Eingebettet in *Assessment* würde jede Modelländerung den gesamten Prüfprozess deployen lassen.

2. Kubernetes-Betriebskonzept:

- *Rollout/Rollback:* **Canary** für *Claim Intake* und *Assessment* (Verkehrsschritte 5%/25%/100%); Rollback automatisch ausgelöst, wenn Fehlerquote > 1% über 5 Minuten. Pflicht: Rollback innerhalb von 10 Minuten geübt.
- *Skalierungsregel:* “Wenn die Queue-Länge in *Claim Intake* > 200 oder die CPU-Auslastung > 70% für 3 Minuten, dann +2 Replicas, bis max. 12.”
- *Konfiguration / Secrets:* Konfig in ConfigMaps; Secrets im Vault (nie im Image); Zugriff role-based; Rotation alle 90 Tage.
- *Monitoring / Alerting:* Zentraler Log-/Metric-/Trace-Standard mit Korrelations-ID pro Schadenfall. Alerts auf Fehlerquote, Latenz (p95), Queue-Länge, fehlende Heartbeats; Eskalation an Service-Owner.
- *Ressourcenlimits:* Pro Container harte Limits (z. B. 500 m CPU, 512 MiB RAM) und Requests bei 70% der Limits. Keine “unbegrenzten” Pods.

Aufgabe 7 – Fallstudie “InsureFast” – Teststrategie & Operating Model

L2 • Anwendung

~ 30 min

YouTube-Suchquery: Microservices Operating Model RACI Plattform Team Incident Management

Musterlösung

1. Teststrategie (TDD):

- *Unit-Tests* für Kernlogik: *Claim Assessment* (Entscheidungsregeln, Begründungslogik), *Fraud Check* (Score-Berechnung), *Payout* (Storno-/Korrekturlogik). TDD-Zyklus *Red–Green–Refactor* verpflichtend für regulatorische Logik.
- *Integrationstests* für Service-APIs: Claim Intake → Assessment, Assessment → Payout, alle Services → Document Management.
- *E2E-Tests* nur für **zwei Kernpfade**: *Happy Path* (Schaden gemeldet → geprüft → ausgezahlt) und *Fraud-Flag-Path* (Schaden gemeldet → Fraud Check schlägt an → manueller Review).
- *Quality Gates*: (1) Alle Tests grün + Security-Scan ohne kritische Findings; (2) Smoke Test nach Deploy, Rollback-Fallback geplant.

2. Operating Model:

Teamzuschnitt: 5–6 Produktteams, jedes Team besitzt 1–2 Services (“you build it, you run it”). 1 Plattformteam (CI/CD-Templates, Observability, Kubernetes-Basis). 1 Security/Compliance-Team (Baselines, Ausnahmen).

RACI Schnittstellenänderung:

	Service-Owner (anrufend)	Service-Owner (gerufen)	Plattform	Security
Änderung initiieren	R / A	C	I	I
Vertragsprüfung	C	R	C	I
Auswirkung abschätzen	C	A	R	C
Freigabe	I	A	C	R / C
Rollback-Plan	R	R	A	I

Incident-Handling: On-call beim Service-Owner; bei Plattformproblemen Eskalation an Plattformteam; Security wird zugeschaltet bei Datenleck-Verdacht. Postmortem innerhalb von 5 Arbeitstagen, blameless.

Release-Freigabepfad: Standard-Release → Service-Owner genügt. Kritische Änderung (Schnittstelle, Datenmodell, Security) → Service-Owner + Plattform + Security.

3. Zwei Entscheidungen unter Unsicherheit:

- *Datenstrategie*: **DB-bleibt + Anti-Corruption-Layer** (kein Big Bang). Begründung: 12 Wochen reichen nicht für Migration unter Audit-Druck; Risiko: Migrations-Bug wäre existenzkritisch. *Frühwarnsignal*: Wenn die Integrationsfehler über den Anti-Corruption-Layer > 5 % steigen oder die Lead-Time pro Änderung sich verdoppelt → Migration einplanen.
- *Release-Frequenz*: **wöchentlich für Nicht-Kernservices, zweiwöchentlich für Kern** (Assessment, Payout). *Frühwarnsignal*: Change Failure Rate > 15 % im Kern → Frequenz reduzieren

oder Quality Gates verstärken.

Aufgabe 8 – Anti-Muster “Microservices-Spaghetti” erkennen und korrigieren

L2 • Anwendung

~ 25 min

YouTube-Suchquery: Microservices Antipattern verteilter Monolith Observability erklärt

Musterlösung**A – Service-Wildwuchs.**

- *Verletztes Prinzip*: “Servicegrenze entlang Capabilities” und “Wirtschaftlichkeit”. 47 Services für 60 Entwickler bedeuten weniger als 1,3 Entwickler pro Service – unten über Owner und Run-Disziplin.
- *Korrektur*: Konsolidieren entlang Capabilities. Faustregel: ein Service braucht mindestens *drei Personen*, die ihn ge- und betreiben können, sonst zu Capability verschmelzen. Kundendaten in einer Hoheit – *Customer Service* – statt vierfach.

B – Verteilter Monolith.

- *Verletztes Prinzip*: “Lose Kopplung” und “unabhängige Deployments”. Wenn jede Änderung mehrere Releases erzwingt, sind die Services nur formal getrennt.
- *Korrektur*: Synchroner Aufrufe zu Events / asynchroner Kommunikation umbauen, wo möglich. API-Versionierung erzwingen (alte Version mindestens eine Iteration weiter laufen lassen). Schnittstellen explizit dokumentieren (OpenAPI/AsyncAPI) und Vertragsbruch-Tests führen.

C – Observability als Nachgedanke.

- *Verletztes Prinzip*: Betriebsdisziplin, “you build it, you run it”.
- *Korrektur*: Zentrale Log-/Metric-/Trace-Standards als Pflicht (Plattformteam stellt sie bereit, nicht jedes Team baut sie nach). *Korrelations-ID* pro Geschäftsfall durch alle Services. Mindest-Dashboard pro Service: Verkehr, Fehlerquote, Latenz, Saturation (“USE/RED-Pattern”). Ziel-MTTR explizit.

D – Rollback-Atrophie.

- *Verletztes Prinzip*: Betriebsdisziplin und Reliability-Engineering. Ein nie geübter Rollback ist im Incident-Fall *kein* Rollback.
- *Korrektur*: Monatliche Rollback-Übung pro Team (Chaos-Light), Rollback-Pfad in jeder Pipeline implementiert und getestet, Rollback-Dauer als SLO für das Plattformteam (z. B. ≤ 10 Minuten). Postmortems erwähnen explizit, ob Rollback möglich gewesen wäre.

Niveau L3 – Management & Entscheidungsarchitektur

Aufgabe 9 – Reliability- & Risk-Framework für eine wachsende Plattform

L3 • Management

~ 35 min

YouTube-Suchquery: SLO Error Budget Change Failure Rate MTTR SRE Framework Senior

Musterlösung

1. SLO-Set (drei SLOs für kritische Services):

SLO	Schwelle	Begründung
Verfügbarkeit	99,9 % / Monat	Branchenüblich für Kundenkernprozesse; entspricht ca. 43 Minuten Ausfall pro Monat.
Latenz p95	≤ 300 ms	Schwelle, ab der Nutzerwahrnehmung spürbar leidet (User Research).
Fehlerquote	$\leq 0,5$ %	Niedrige Schwelle, weil Fehler in der Kernlogik direkt in Reklamationen und Audit-Findings durchschlagen.

2. Error Budgets: Wenn die Verfügbarkeit 99,9 % / Monat als SLO definiert ist, beträgt das *Error Budget* 0,1 % / Monat (~ 43 Minuten). Solange das Budget *nicht* aufgebraucht ist, dürfen Teams releasen. Wird das Budget aufgebraucht, gilt:

- Release-Stop für den betroffenen Service – ausgenommen sicherheitskritische Hotfixes.
- Plattform-Review innerhalb von 5 Arbeitstagen mit dem Service-Owner: Ursache, Maßnahme, Lernen.
- Erst nach 14 Tagen ohne weitere Verletzung wird der Release-Stop aufgehoben.

Sinn: Speed und Reliability werden *verhandelbar* – kein bloßer Appell.

3. Risk-Framework (fünf Risiken):

Risiko	Impact	Whk.	Prävention
Servicegrenze falsch geschnitten	5	3	Capability-Mapping vor Schnitt, Architekturreview für irreversible Entscheidungen.
Secret im Code	5	3	Secret-Scanning als Quality Gate; Vault verpflichtend.
Rollback nicht geübt	4	4	Monatliche Rollback-Übung, Rollback-Dauer als SLO.
Observability-Lücken	4	4	Mindest-Dashboard verpflichtend, Korrelations-ID Standard.
Tooling-Lock-in	3	3	Offene Standards (OpenAPI, BPMN-XML, Helm), Export-Anforderung in Tool-Auswahl.

4. Anti-Gaming-Mechanik:

- *Kopplung 1:* Change Failure Rate gilt nur dann als “erfüllt”, wenn gleichzeitig der *Service-Health-Index* (interner Composite-Wert aus Fehlern + Eskalationen + Retouren) sich nicht ver-

schlechtert. So bringt das Klein-Halten von “Changes” keinen scheinbaren Erfolg.

- *Kopplung 2*: Postmortem-Pflicht für jeden *Kundenwahrnehmbaren* Incident; ein nicht durchgeführter Postmortem zählt als Change Failure mit doppelter Gewichtung. So lohnt es sich nicht, Incidents zu vertuschen.

Aufgabe 10 – Governance leicht, aber wirksam

L3 • Management

~ 30 min

YouTube-Suchquery: Architecture Governance Principles Senior Lightweight Service Standard

Musterlösung**1. Acht Architekturprinzipien:**

1. Servicegrenzen folgen *Business Capabilities*, nicht Datenbanktabellen oder Org-Charts.
2. Jeder Service hat *einen* Owner; Ownership ist an die Funktion gekoppelt, nicht an Personen.
3. Kein Secret im Code; Credentials kommen ausschließlich aus dem Vault.
4. Jede öffentliche Schnittstelle ist als OpenAPI/AsyncAPI dokumentiert und versioniert; Vertragsbrüche werden in der Pipeline erkannt.
5. Schreibende Operationen sind *idempotent* (Operations-ID), Konsumenten dürfen sicher wiederholen.
6. Jeder Service stellt ein Mindest-Dashboard (Verkehr, Fehlerquote, Latenz, Saturation) bereit und schreibt strukturierte Logs mit Korrelations-ID.
7. Releases gehen über Canary oder Blue/Green; Rollback ist innerhalb von 10 Minuten möglich und monatlich geübt.
8. Datenhoheit ist eindeutig: ein Service “besitzt” eine Capability, Daten anderer Services werden über Schnittstellen oder Events bezogen – nicht via geteilter Datenbank.

2. Schnittstellenstandards (zwei):

- *OpenAPI / AsyncAPI als Single Source of Truth* für synchrone bzw. asynchrone Schnittstellen – jeder Service veröffentlicht seine Schemas im Service-Katalog.
- *Event-Schema-Registry* (z. B. Avro/JSON-Schema) mit Versionierung und Rückwärtskompatibilitäts-Prüfung als Quality Gate.

3. Review-Regel:

- *Architektur-Review verpflichtend bei:* neuer Service, neue Schnittstelle nach außen, Datenhoheits-Änderung, Plattformwechsel, Sicherheits-Baseline-Ausnahmen.
- *Nicht erforderlich bei:* interne Refactorings, neue interne Endpoints, Test-/Toolwechsel innerhalb des Teams.

Faustregel: Review nur bei *teuren oder irreversiblen* Entscheidungen – alles andere kostet mehr Zeit als es schafft.

4. Eskalationspfad (vier Stufen):

1. Service-Owner und Plattform-Lead versuchen Einigung (max. 2 Arbeitstage).
2. Bei Patt: Architekturboard (Plattform-Lead + zwei rotierende Senior-Engineers + Security).
3. Bei strategischem Dissens: Head of Platform / Head of Product.
4. Bei strategischer Verantwortung: CTO. Entscheidungen sind dokumentiert (ADR – Architecture Decision Record).

Aufgabe 11 – Transferprojekt – Digitale Prozessarchitektur als Betriebs- und Entscheidungsarchitektur

L3 • Management

~ 45 min

YouTube-Suchquery: Microservices Operating Model Decision Architecture Senior Platform CIO

Musterlösung

Musterlösung als Entscheidungsarchitektur – andere Konfigurationen sind möglich, solange sie als Architektur (nicht als Liste) gedacht sind.

1. Top-10-Entscheidungsarchitektur:

Entscheidung	Wer entscheidet?	Optionen / Kriterien
Servicegrenzen	Architekturboard Service-Owner	+ Capability-Schnitt; max. 20 Services in Phase 1.
Datenhoheit	Architekturboard	Eine Capability = eine Datenhoheit; kein Shared-DB.
API-/Event-Standards	Plattform-Lead	OpenAPI/AsyncAPI verbindlich; Schema-Registry.
SLOs	Service-Owner + Plattform	Standard-Set (Verf./Lat./Err) je Klasse.
Release-Policies	Plattform-Lead + Security	Standard-Releases via Canary; Kern-Release zweiwöchentlich.
Security-Baselines	CISO + Plattform	Vault, Least Privilege, SBOM, Secret-Scan.
Observability-Standard	Plattform	Zentraler Log-/Metric-/Trace-Stack; Korrelations-ID Pflicht.
Incident-Governance	Head of Platform	On-call beim Service-Owner; Eskalation an Plattform.
Ausnahmeprozess	Architekturboard	Schriftliche Ausnahme, Ablaufdatum, Reviewer.
Toolchain-Standard	CTO	1 – 2 Tools pro Kategorie; Export-Pflicht.

2. Operating Model & RACI:

- *Produktteams (Build + Run):* Service-Owner, Tag-Betrieb, Releases, On-call, Postmortems.
- *Plattformteam:* CI/CD-Templates, Observability, Kubernetes-Basis, Rollback-Mechanik, Schema-Registry.
- *Security/Compliance:* Baselines, Audit-Beweise, Ausnahmen, SBOM-Pflicht.
- *Architekturboard:* nur teure/irreversible Entscheidungen; max. 1 h pro Sitzung, alle 2 Wochen.
- *Eskalation bei Speed vs. Risk:* Service-Owner → Plattform-Lead → Head of Platform → CTO.

3. Reliability- & Risk-Framework:

- SLOs pro Service (Standard-Set), Error Budgets, Change Failure Rate ($\leq 10\%$), MTTR (≤ 60 min).

- Anti-Gaming-Kopplung: Kosten-/Geschwindigkeits-KPIs gelten nur, wenn *Service-Health-Index* nicht degradiert (s. Aufgabe 9).

4. Governance leicht, aber wirksam: max. 8 Architekturprinzipien (s. Aufgabe 10). Standard-Templates für CI/CD, Helm/Deployment, Mindest-Dashboard, Postmortem.

5. Roadmap (16 Wochen):

- *MVP (Woche 1–8):* Plattform-Templates fertigstellen; zwei Pilotteams onboarden; SLO-Standard-Set live; drei Services extrahieren; Canary-Pipeline verfügbar.
- *Scale (Woche 9–16):* weitere fünf Teams; wöchentliches Release; Error-Budget-Mechanik aktiv; Architekturboard mit ADR-Praxis.
- *Messkonzept:* Outcomes (Lead Time, Change Failure Rate, MTTR, Audit-Findings) – keine Aktivitätsmetriken (“wie viele Tickets bearbeitet”).

6. Zwei Entscheidungen unter Unsicherheit (Pflicht):

1. *Welche drei Services zuerst extrahieren?*

Wahl: *Claim Intake*, *Fraud Check*, *Notifications*. Begründung: Hoher Lasthebel (*Intake*), klare Capability (*Fraud*), wenig Datenkopplung (*Notifications*). *Verworfen Alternative:* *Payout* zuerst – zu hohes Audit-/Finance-Risiko, kein guter Lern-Service. *Frühwarnsignale:* Wenn Lead-Time für *Intake* sich nach 4 Wochen verdoppelt oder Fraud-Score-Genauigkeit sinkt → Schnitt überprüfen.

2. *Welche Release-Policy für Kernprozesse?*

Wahl: *Zweiwöchentlich* + *Canary 5 % / 25 % / 100 %*, Rollback automatisch bei Fehlerquote > 1 %. Begründung: Kernprozesse = auditrelevant + finanzkritisch; wöchentlich nicht durchhaltbar bei aktuellem Reifegrad, monatlich wäre zu langsam. *Guardrails:* kein Release am Freitag; Security-Approval Pflicht; Rollback-Übung monatlich. *Frühwarnsignal:* Wenn Change Failure Rate im Kern > 15 % in 2 Iterationen → Frequenz zurück auf monatlich.

Anschluss an Strategie: Auditierbarkeit (Compliance), Time-to-Market (Peak-Saison), Skalierbarkeit (mehrere Standorte). Die Plattform ist kein IT-Projekt, sondern ein *steuerbares System*, das Entscheidungen, Risiko und Verantwortung sichtbar macht.

Abschluss

Die Musterlösungen sind eine Diskussionsbasis. Wenn Ihre eigene Lösung anders ist, fragen Sie: *Habe ich andere Annahmen – oder einen anderen Service-Schnitt angelegt?* Beides ist legitim, solange die Logik nachvollziehbar bleibt.

Nächster Schritt

Bringen Sie Ihre eigenen Lösungen in die Coaching-Sitzung mit. Im Fokus stehen drei Fragen: Welche Annahmen haben Sie gemacht? Welche Zielkonflikte (Speed vs. Stabilität, Autonomie vs. Standardisierung, Aufwand vs. Auditierbarkeit) haben Sie sichtbar gemacht? Welche Entscheidung haben Sie getroffen – und würden Sie sie auch verantworten, wenn die nächste Reorganisation kommt?